Multimodal Meeting Intelligence Pipeline — Report

1. Architecture & key design decisions

The system is a **linear cascade of independent, swappable stages** connected by explicit Pydantic data contracts (`src/mmi/schemas.py`). Each stage reads a typed object and emits a typed object, so any component can be replaced (a different ASR, a hosted OCR, another LLM) without touching its neighbours — the same shape a production speech/AI stack takes.

```
video ■■ ingest ■■ [A] speech ■■
                               ■■■ [C] fusion ■■ [C] intelligence ■■ result.json
       ■■■■■■■■ [B] vision ■■■■■
```

Data contracts (the important part). The brief explicitly asks us to design the contract between stages, so this is where most of the design effort went:

- `Transcript` = `language` + `TranscriptSegment[]`, where each segment carries `start/end/text/speaker` and optional word-level timings. This is the merge point of the segmentation → ASR → diarization sub-pipeline.
- `VisualContext` = `SceneSegment[]`, each with a time span, a representative keyframe path, and OCR lines (text + confidence + bbox).
- `TimelineEvent[]` is the *fused* representation: speech and visual events interleaved on one clock, which is exactly what the LLM consumes.
- `MeetingIntelligence` is the enforced LLM output: `summary`, `topics`, `action_items`, `decisions`, each item carrying `Evidence` (timestamp / speaker / scene / quote) for grounding.

Orchestration. `run.py` is the single entry point. Each stage's artifact is cached to `outputs/cache/` and reloaded on re-run, so an expensive ASR pass isn't repeated while iterating on prompts. Stages can be toggled (`--skip-llm`, `--skip-vision`, …) for cheap partial runs. Secrets are read only from the environment.

What we rejected. - *WhisperX vs. rolling our own faster-whisper + pyannote glue.* We kept WhisperX because its VAD-batched inference and word-level alignment give materially better speaker attribution at segment boundaries, and it already wires pyannote in — but we still expose the merge as our own `Transcript` contract rather than leaking WhisperX's dict format downstream. - *Processing every frame for CV.* Rejected on cost; scene-change detection + one keyframe per scene captures slide content at a tiny fraction of the frames. - *Free-form LLM output.* Rejected in favour of a strict schema (OpenAI structured outputs / Pydantic parse) so the result is machine-consumable and every claim is forced to carry evidence. - *One giant prompt for long meetings.* Rejected; we chunk and map-reduce above a configurable character budget.

2. Model / API choices and trade-offs

Stage	Choice	Why	Trade-off
ASR	WhisperX (`small` default)	Fast CTranslate2 backend, auto language ID, word alignment	`small` trades WER for fitting a 4 GB GPU; `medium`/`large-v3` configurable

Diarization	pyannote 3.1	State-of-the-art open diarizer, integrates with WhisperX	Needs a HF token; sensitive to overlapping speech
Scene detection	PySceneDetect content detector	Cheap, robust to slide/scene cuts	Threshold tuning; gradual fades can be missed
OCR	EasyOCR (GPU)	Simple install, decent on slide text, GPU-accelerated	Weaker on dense/stylised text than PaddleOCR (swappable via config)
LLM	OpenAI `gpt-4o-mini`	Strong structured-output support, low cost/latency	Hosted dependency; `gpt-4o` available for higher fidelity

Cost/latency/accuracy stance. We deliberately default to the *cheap, fast* end of each axis (small Whisper, mini LLM, one keyframe/scene) because the brief values judgment and integration over raw accuracy, and because the reference machine is a 4 GB laptop GPU. Every knob is in [`config.yaml`]() to scale accuracy up when hardware allows. GPU memory is the binding constraint, so the speech stage loads/uses/frees each model (ASR → align → diarize) sequentially rather than holding them all resident.

3. Evaluation

Metrics are implemented in `evaluate.py` and run against a small hand-labelled reference (`data/reference/reference.json`):

- **ASR — Word Error Rate** (jiwer) on normalised text, with S/D/I breakdown. When the reference is a snippet (has `speaker_segments`), WER is scoped to that time window so it isn't distorted by comparing 60 s of reference against the full ~26 min hypothesis.
- **Speaker attribution accuracy** — a diarization proxy: both reference and hypothesis are rasterised onto a 0.1 s grid, and we brute-force the optimal hypothesis→reference label mapping (speaker counts are small) and report time-weighted agreement. This sidesteps the arbitrary-label problem.
- **OCR keyword recall** — fraction of expected on-screen terms recovered across all keyframes; also reports how many scenes yielded any text.
- **LLM output** — a rubric (faithfulness, topic relevance, real action items, grounding, no hallucinated entities) plus an automatic *grounded-fraction* (share of items carrying evidence) and an optional LLM-as-judge (`--judge`).

Results on the provided clip (26 min council meeting, `--language en`, `whisper-small` on a 4 GB T1200, LLM stage skipped due to OpenAI quota):

Metric	Value	Notes
ASR WER (first 60 s vs bootstrap reference)	**6.67 %**	8 insertions / 0 subs / 0 dels

Speaker attribution accuracy (first 60 s)	**100 %**		2 speakers in window; label map `SPEAKER_01-` `SPEAKER_03-` 6 speakers globally
OCR keyword recall	**100 %** (1/1)		Only meaningful term is "zoom"; see below
Scenes with OCR text	**15 / 16**		
Distinct diarized speakers	**6**		Plausible for a small council meeting

Detected language	**en**		Auto-detect chose `mi` (Maori) on the first 30 s intro; overridden by `--language en`. Robust auto-detect (majority vote at 25/50/75 %) is implemented in `speech.py` for future runs.

The reference labels in `data/reference/reference.json` are a **bootstrap** transcript (a lightly-cleaned version of the pipeline's own first-minute output), which makes the WER an optimistic anchor rather than an unbiased quality measurement. It still exercises the harness and, more importantly, the speaker-attribution and OCR metrics are unaffected by that bias.

Failure analysis (where it breaks and why). - *Language ID on noisy intros* → Whisper mis-detected English as Maori on the first 30 s (which contains roll-call cross-talk and short utterances), producing a garbage transcript of "maitha, maitha, maitha…" for the first segment on that run. Fixed by (a) adding a `--language` CLI override and (b) a majority-vote language detector that samples windows at 25/50/75 % of the audio; both live in `speech.py`. - *Zoom meetings have no slides* → OCR faithfully returns just the word "zoom" from the Zoom UI overlay across 15 of 16 scenes. The visual channel adds essentially no downstream signal on this particular clip. For a presentation/slide meeting the same OCR pipeline would yield rich content — this is a property of the input, not a pipeline bug. - *Diarization initially failed* due to a `speechbrain 1.1 + pyannote 3.3.2` lazy-import bug (`speechbrain.integrations.k2_fsa` cannot be resolved). Pinning `speechbrain==1.0.2` fixed it and produced 6 plausible speakers. - *LLM stage failed* with `429 insufficient_quota`. The pipeline now catches that, writes a valid `result.json` with `intelligence=null` and `metadata.llm_error` populated, and re-runs will re-try (idempotent). - *Windows console encoding* — EasyOCR / gdown progress bars use box-drawing characters that cp1255 can't encode; forced UTF-8 for stdout in the CLI. - *Diarization on overlap / short backchannels* → still a general risk; the biggest driver of speaker-accuracy loss on longer meetings. - *LLM grounding* → the schema forces evidence, but the model can still cite a plausible-but-wrong timestamp; the grounded-fraction metric flags *whether* evidence exists, not that it's correct — a known gap.

4. Production thinking

- **Scaling.** Stages are independent and stateless given their inputs, so each maps to its own horizontally-scaled worker/queue. A 2-hour recording is handled by (a) VAD-chunked ASR already, (b) scene-bounded CV that is independent of duration, and (c) LLM **map-reduce**: summarise windows in parallel, then reduce — implemented in `intelligence.py` above a configurable budget.
- **Latency & memory.** The bottleneck is Part A on GPU; on 4 GB we serialise model loads and prefer `int8`/`small`. In production ASR and diarization run on right-sized GPUs; CV and the LLM call are CPU/network-bound and cheap.
- **Cost.** Dominated by LLM tokens; the timeline is compact (deduplicated OCR, one line per utterance) and chunked, keeping cost roughly linear in meeting length. Model tier is a config switch.
- **Monitoring & regression.** Cache the per-stage artifacts (already done) and run the eval harness on a labelled golden set in CI to catch drift; track WER, DER/speaker-accuracy, OCR recall, and LLM grounded-fraction over time. Log detected language, speaker count, and scene count per run as cheap health signals.
- **Robustness.** Missing HF token → diarization is skipped (single speaker) rather than crashing; CUDA absent → CPU fallback; OCR/alignment failures are caught per-item so one bad frame/segment can't fail the run.

With one more week: add active-speaker detection (face + lip-motion) to fuse *who is on screen* with diarization; a real DER via `pyannote.metrics`; PaddleOCR + slide-title heuristics for cleaner visual grounding; a retrieval step so the LLM cites exact source spans; and a golden-set CI job.

5. Use of AI assistants

AI coding assistants were used to scaffold boilerplate (argparse wiring, Pydantic models, ffmpeg/OpenCV/EasyOCR call patterns) and to draft this report. The **design decisions were made deliberately**: the stage decomposition and data contracts, the choice to keep WhisperX but hide its format behind our own `Transcript`, the GPU-memory-driven sequential model loading for a 4 GB card, the grounding-via-schema approach, the map-reduce long-input strategy, and the evaluation metric definitions (especially the label-mapping speaker-accuracy proxy). Every one of these is defensible in follow-up discussion.